

Privacy-Preserving Two-Party RBF Kernel SVM Training Based on Neat and Accurate Secure Exponentiation

Qingyu Mo, Wenyan Wu, Jingwei Chen

Abstract—Privacy-preserving machine learning (PPML) is a powerful tool for multiple parties to collaboratively train a model or perform model inference without exposing their private data in the context of Internet of things. A key challenge in PPML is the efficient evaluation of non-polynomial functions. In this work, we propose NASE, a neat and accurate secure exponentiation protocol for radius basis function (RBF) kernel evaluation. Leveraging the property of the RBF kernel, NASE enjoys a lightweight construction that reduces computation overhead by up to $1.65\times$ and communication overhead by up to $3.97\times$ compared to SIRNN, the prior SOTA framework for secure exponentiation published in *IEEE S&P 2021*.

Taking NASE as the foundation stone, we propose a privacy-preserving two-party kernel SVM training protocol. Based on BFV scheme and MPC technique, we introduce group-batch sampling for sampling in ciphertext and propose the partial rotation method tailored to our scenario to optimize dot product computation. Additionally, we propose an error-tolerant *DReLU* protocol for secure sign evaluation of secret sharings over a prime field that reduces the communication cost by around $\frac{1}{3}$ compared to the existing method. Our protocol achieves model accuracy comparable to plaintext training according to experiments on real-world datasets, and an order-of-magnitude reduction in both communication and computation overhead is attained compared to the previous work.

Index Terms—privacy-preserving machine learning, medical Internet of things, support vector machine, kernel methods

I. INTRODUCTION

SUPPORT vector machine (SVM) is a classic and robust supervised learning algorithm in classification tasks. In the context of the Internet of things (IoT)—especially in medical IoT (MIoT) scenarios—it has found extensive applications such as disease diagnosis, patient monitoring, and health data analysis [1], [2], [3]. In such scenarios, data is often generated by wearable sensors, medical imaging devices or hospital information systems and held by different institutions. In this work, we focus on the two-party setting, where two institutions or hospitals wish to collaboratively perform SVM training or inference using their data.

A fundamental challenge confronting collaborative SVM training or inference in MIoT lies in the sensitivity of the data. The medical data is typically associated with the health status, identity and other private attributes of individuals. Disclosing such data for SVM tasks can lead to privacy leakage. There are also regulations like GDPR and HIPAA that prohibits unauthorized sharing of personal health data. Convention encryption techniques can protect data during transmission, but they also make the data unusable for joint SVM tasks.

To address this issue, recent advances in privacy-preserving machine learning (PPML) have introduced cryptographic techniques that allow joint model training without compromising data privacy. The majority of techniques for secure computation includes homomorphic encryption (HE) [4], [5], [6] and multi-party computation (MPC) [7], [8], [9]. Concerning HE-based solutions, a representative work for secure SVM tasks in the MIoT context is Wang et al. [10], which employs DT-PKC [11], a partially HE scheme, achieving accuracy comparable to the plaintext SVM model. The advantage of HE is that it is efficient in arithmetic operation, and modern leveled HE schemes such as BFV [4] further support SIMD batching, enabling parallel evaluation on encrypted vectors. However, HE remains generally inefficient in non-polynomial operation. MPC is more flexible for boolean operations [7], but it usually incurs heavier communication cost. Recent work [12], [13], [14] suggests combining BFV-based HE and MPC techniques for efficient secure computation. Such fusion takes advantage of HE and MPC, ensuring efficiency in both polynomial and boolean operations. This hybrid approach has emerged as a promising solution for scalable and practical secure computation.

Despite extensive progress in two-party privacy-preserving SVM research based on HE and MPC, two major challenges remain underexplored and lack efficient solutions: the first is the secure evaluation of the **radius basis function (RBF)** kernel function, and the second is the design of practical protocols for privacy-preserving SVM training.

As for the first challenge, we first discuss what is the kernel function. The kernel function enables data to be implicitly mapped into high-dimensional feature space. It has been a topic of interests in machine learning [15], [16], [17], [18] nowadays. Among the commonly used kernel functions—linear, polynomial and RBF kernel—RBF kernel is particularly effective in capturing non-linear relationships between data. Given two data point $\mathbf{x}_0$, $\mathbf{x}_1$ and a hyper-parameter γ , the RBF kernel is denoted as $e^{-\gamma\|\mathbf{x}_0-\mathbf{x}_1\|^2}$.

Concerning the secure evaluation of kernel functions for privacy-preserving SVM tasks, the computation of linear and polynomial kernel functions are relatively easy since only polynomial operations are needed, while the evaluation of RBF kernel poses greater challenges. When $\mathbf{x}_0$ and $\mathbf{x}_1$ are distributed in different parties which is a common case for secure kernel SVM task, computing the RBF kernel essentially becomes the two-party secure exponentiation problem with a secret floating-point exponent. HE and MPC frameworks typically operate over fixed-point integers and lack native support for computing such non-linear and floating-point function. While RBF kernel outperforms linear and polynomial

kernel in handling non-linear relationships between data and provides stronger security against malicious attack [19] for model retrieval, the necessity to make use of RBF kernel and its incompatibility to secure computation frameworks has been a dilemma.

As for the previous work to securely compute the RBF kernel, some relies on polynomial approximation. Random Fourier feature (RFF) [20] and Taylor extension are usually employed, by which an offline RBF kernel machine inference could be achieved combining HE technique. However, a major limitation of approximation methods is that they are only efficient and precise within a small range, hindering their robustness in real-world applications.

A more practical approach is to compute the exponential function in plaintext and securely retrieve it. State-of-the-art protocols following this paradigm, such as SIRNN [21] and [22], achieve high computational accuracy and demonstrate practical efficiency. Nevertheless, specific design choices in these protocols introduce bottlenecks that hinder further performance improvements.

The second challenge is secure SVM training in two-party settings. Unlike the inference phase, SVM training involves iterative optimization with more complex operations, making it harder to implement by secure computation technique. A suitable algorithm for secure SVM training could be PEGASOS [23], since each iteration in it only involves relatively simple operations. Nevertheless, when adapting PEGASOS to the BFV-MPC setting directly, PEGASOS encounters two major efficiency bottlenecks. The first lies in the mini-batch sampling process of PEGASOS. The use of BFV scheme requires the data to be encrypted in BFV ciphertexts. As a result, operations involved in mini-batch sampling-such as random indexing, reshuffling, and batch formation-become difficult to perform, since ciphertexts are encoded in fixed structures that constrains data access. In addition, secure training with PEGASOS must cope with the heavy cost of repeated high-dimensional matrix-vector secure dot products in BFV. These difficulties together make secure SVM training expensive in MIoT scenarios.

With respect to the challenges mentioned above, we make the following contributions in this work:

- We propose NASE, a neat and accurate secure exponentiation protocol based on MPC for secure RBF kernel evaluation. Based on the fact that the exponent of the RBF kernel function is always non-positive, NASE substitutes the millionaire protocol merely by 1 OT in its computation. Thanks to its lightweight construction, NASE enjoys lower computation cost and reduced communication overhead compared to SIRNN and [22]. On average, NASE achieves $1.60\times$ runtime speedup and $3.24\times$ reduction in communication overhead on our simulated dataset, and $1.30\times$ runtime speedup with $2.57\times$ reduction in communication overhead on our real-world datasets compared to SIRNN. We also provide an exact analysis on the error bound of NASE to discuss its accuracy. NASE is also applicable to other privacy-preserving tasks besides RBF kernel computation, e.g., secure Sigmoid and Tanh function evaluation.

- Built on NASE, we propose a secure two-party RBF kernel SVM training protocol based on MPC and BFV scheme that is able to perform training on a dataset of several hundred samples within seconds and several thousand samples within minutes. Specific to the attribute of BFV ciphertext, we have made two optimizations to improve the efficiency of our training protocol: a so-called group-batch sampling strategy that enables easy secure batch sampling and more efficient packing strategies for the kernel matrix, and a secure dot product protocol that reduces the number of ciphertext rotations by several times. We also propose a new construction of *DReLU* protocol (used to securely check the sign of a secret sharing) for the secret sharing over a prime field whose communication cost is reduced by around $\frac{1}{3}$ of [12]. Compared to [24], we achieve an order-of-magnitude reduction in both communication and computation overhead, and our model achieves 4% higher accuracy on average.

II. RELATED WORK

A. Secure RBF Kernel Function Evaluation

For the secure two-party RBF kernel evaluation task, our goal is to obtain the evaluation result in the form of secret sharing given private inputs from both parties. Existing work on secure RBF kernel evaluation mainly relies on approximation and direct computation methods.

1) Approximation. The majority of approximation-based solutions rely on RFF [25], [26] and Taylor extension [27]. Their common idea is to convert the RBF kernel evaluation task to a linear kernel computation one with extended feature length. A general problem of approximation-based methods is that their accuracy and efficiency degrade significantly as the range of the exponent increases.

2) Direct Computation. Concerning methods to calculate the exponential function directly, a sensible strategy is to compute it using the exponent masked by a random number and securely de-mask it. The protocols in [28] and [29] follow such idea, but the random mask they rely on is constructed under a small security parameter, resulting in insufficient security guarantees. Xie et al. [24] proposes a two-party RBF kernel computation protocol based on MPC technique, but we discover that the protocol has a non-negligible chance to fail when the additive sharing of the exponent wraps around the modular of its ring. A secure exponentiation protocol is proposed in [21] on which the work of Wu et al. [30] is based to compute the RBF kernel for secure SVM inference. Given $\gamma||x - y||^2$ scaled and additively shared by two parties, it precomputes the exponential function of all possible input values in plaintext and makes use of secure LUT [31] to obtain the corresponding exponential result. Although this protocol is highly accurate, it suffers from the large communication overhead brought by LUT. The exponentiation protocol proposed by Kelkar et al. [22] makes most of its sub-protocols non-interactive at the cost of operating over a large ring. However, the use of the large ring significantly increases the cost of its only interactive sub-protocol, MTA protocol, potentially adding up the total communication and computation burden.

B. Secure SVM Training

The two-party secure inference stage of SVM has been exhaustively studied in previous research [32], [33], [29], [28], [34], [35], [36], but there is only limited amount of research on the training phase. Some work [37], [38] uses gradient descent that only involves one comparison operation in each iteration to securely solve the primal problem, but they are not applicable to kernel tricks. Liu et al. [39] securely implements sequential minimal optimization (SMO) [40] which is deemed the most powerful optimizer for SVM training in plaintext. But it should be noted that a single iteration of SMO requires six comparison operations and one division operation, making it unsuitable to general secure computation framework.

Some research opts to optimizing simpler SVM objective functions for training kernel SVM with fewer non-polynomial evaluations. Park et al. [41] suggests solving a variant of standard SVM, LSSVM, in a fully homomorphic encryption manner. However, we discover that their gradient descent method heavily relies on the proper choice of learning rate which is not easy to fine-tune securely. Another modified objective function that is frequently utilized for privacy-preserving SVM training [10], [42], [24] is the hinge loss, and it can be effectively solved by sub-gradient descent. The sub-gradient descent only takes one non-polynomial operation per iteration. Shalev-Shwartz et al. [23] proposes a powerful solver for the hinge loss called PEGASOS that automatically assigns a proper step size for gradient descent in each iteration, and it has been proven in [42] through theoretical analysis that SMO is inferior to PEGASOS in terms of secure computation.

III. PRELIMINARIES

In this section, we introduce the notations involved in this paper and the prior knowledge needed for our protocol.

A. Notations

We denote the two participating parties as P_0 and P_1 . Unless specifically stated, we define the symbol of vectors in bold and matrices in capital letter. The subscript of a plaintext data indicates which party it belongs to. We define $\cdot$ as the multiplication operator for single numbers and dot product operator for vectors and matrices. The operator $\times$ is defined as the Hadamard product. The i -th element of vector $\mathbf{x}$ is represented by $\mathbf{x}[i-1]$, and the i -th row of matrix X is represented by $X[i-1]$. The element corresponding to the i -th row and j -th column of matrix X is denoted as $X[i-1][j-1]$, and we define its sub-matrix from i -th to j -th row and from k -th to z -th column as $X[(i-1):(j-1)][(k-1):(z-1)]$. The input of some protocols and operators in this paper is single numeric input, but these protocols are also able to handle vector and matrix input. When the input is a vector or matrix, it means that the protocols and operators are applied individually to each element and result in a vector or matrix as the output.

Our paper extensively uses signed integers. We define $\mathbb{Z}_p$ as the set of integers in $[-\lfloor \frac{p}{2} \rfloor, \lfloor \frac{p}{2} \rfloor]$ when p is odd or in $[-\frac{p}{2}, \frac{p}{2}]$ when p is even. The modulus operator, $\text{mod } p$, is to project an arbitrary integer into $\mathbb{Z}_p$. For a signed number $x \in \mathbb{Z}_p$, its most significant bit is denoted as $MSB(x)$. We denote its

secret sharing as $[x]^p$, and $[x]_i^p \in \mathbb{Z}_p$ indicates its secret share held by P_i . When x is a float number, $[x]^p$ indicates rounding x and sharing it. The symbol of $[\mathbf{x}]^p$ or $[X]^p$ denotes a vector or matrix shared between P_0 and P_1 .

The boolean secret sharing is denoted as $\langle x \rangle$. For boolean operators, we use $\wedge$ and $\oplus$ to respectively represent the AND and XOR operation. We use $\mathbf{1}\{b\}$ as the indicator function that outputs 1 when b is true and 0 otherwise. We denote the bitwise right shift operator as $\gg$.

B. Oblivious Transfer

Given two l -bit inputs x^0 and x^1 held by Alice and a choice index $i \in \{0, 1\}$ held by Bob, a 1-out-of-2 oblivious transfer (OT), denoted as $\binom{2}{1}$ -OT $_l$, is to let Bob retrieve x^i while ensuring that Bob does not learn anything besides x^i and Alice does not know i . The communication cost given l -bit inputs is $\lambda + 2l$ bits to generate an OT relation using OT extension technique [43]. As for $\binom{N}{1}$ -OT $_l$, namely 1-out-of- N OT, we follow [44] to generate it.

C. Secret Sharing

For a value $x \in \mathbb{Z}_p$, its arithmetic secret sharing $[x]_0^p$ and $[x]_1^p$ is respectively held by P_0 and P_1 , and it satisfies $[x]_0^p + [x]_1^p = x \text{ mod } p$. Boolean secret sharing is a special case of arithmetic sharing when $p = 2$. We define common secret sharing functionalities that are used in this work for secure two-party computation as follows.

- $F_{MUX}(\langle x \rangle, [y]^p)$, outputs $[z]^p$ such that $z = x \cdot y$.
- $F_{B2A}(\langle x \rangle, p)$, outputs $[x]^p$.
- $F_{AND}(\langle x \rangle, \langle y \rangle)$, outputs $\langle z \rangle$ such that $z = x \wedge y$.
- $F_{CT}(a_0, a_1, l)$ where a_i is the private input from P_i and l is the bit length of a_0 and a_1 , outputs $[a_0 \cdot a_1]^{4^l}$.
- $F_{MCT}(A_0, A_1, p, h)$. Given two unsigned matrices, $A_0 \in \mathbb{Z}^{a \times b}$ from P_0 and $A_1 \in \mathbb{Z}^{b \times c}$ from P_1 , the maximum values of the elements in A_0 and A_1 is denoted as p and q . The protocol takes A_0, A_1, p and h that satisfies $h > 2bq$ as input, and outputs $[A_0 \cdot A_1]^{ph}$.
- $F_{Mil}(a_0, a_1, l)$ where non-negative integers a_0 from P_0 and a_1 from P_1 have bit length l , outputs $\langle \mathbf{1}\{a_0 > a_1\} \rangle$.

The construction and security of the above protocols follow [12], [21], [7]. We refer interested readers to these works for further details.

D. Lattice-based Homomorphic Encryption

Homomorphic encryption (HE) is a cryptographic primitive that enables computations on encrypted data without knowledge of the decryption key. Our work uses the Brakerski-Fan-Vercauteren (BFV) scheme [4] to realize secure polynomial computation. Compared to other HE schemes, BFV supports SIMD operations, enabling operations on vectors over a specified prime field. We define N as the polynomial degree of the BFV ciphertext and briefly describe the basic functionalities in BFV as follows.

- **Key generation:** $\text{keygen}(N)$, outputs the public key pk , the secret key sk , the Galois key gk and the relinearization key rk .

- **Encryption:** $enc(\mathbf{a})$, encrypts a vector $\mathbf{a}$ of length N .
- **Decryption:** $dec(enc(\mathbf{a}))$, outputs $\mathbf{a}$.
- **Homomorphic addition:** We refer $enc(\mathbf{a}) \boxplus enc(\mathbf{b})$ and $enc(\mathbf{a}) \boxplus \mathbf{b}$ to homomorphic addition which yields $enc(\mathbf{a} + \mathbf{b})$.
- **Homomorphic multiplication:** We refer $enc(\mathbf{a}) \boxtimes enc(\mathbf{b})$ and $enc(\mathbf{a}) \boxtimes \mathbf{b}$ to homomorphic multiplication that yields $enc(\mathbf{a} \times \mathbf{b})$.
- **Rotation:** $rot(enc(\mathbf{a}), s)$, outputs $enc(\mathbf{b})$ where $\mathbf{b}$ indicates $\mathbf{a}$ left-shifted by s steps.

E. RBF Kernel SVM

SVM is a classic model in machine learning. The linear SVM is to find the hyperplane that best separates the two classes of data while ensuring the margin between the hyperplane and the closest data points is as large as possible. Combining kernel tricks, SVM can handle non-linearity of data. Given the input data $\mathbf{x}_0$ and $\mathbf{x}_1$, The RBF kernel function is defined as follows:

$$K(\mathbf{x}_0, \mathbf{x}_1) = e^{-\gamma \|\mathbf{x}_0 - \mathbf{x}_1\|^2}, \quad (1)$$

where γ is the hyper-parameter to determine the influence of data points on the computation of kernel function. Our secure training protocol trains the SVM model by optimizing the hinge loss function, and its design is motivated by PEGASOS [23]. Given X the training set and its corresponding label $\mathbf{y}$, $batch$ the batch size, $iter$ the maximum number of iterations, c a parameter to control the toleration to wrong classifications and α the parameter to be learned, the workflow of PEGASOS is shown in Algorithm 1.

Algorithm 1: Mini-batch PEGASOS.

Input: $X \in \mathbb{R}^{m \times n}$, $\mathbf{y}$, γ , c , $iter$, $batch$

Output: α

- 1 Initialize $\alpha \in \{0\}^m$.
 - 2 Compute RBF kernel matrix $K \in \mathbb{R}^{m \times m}$ where $K(i, j) = e^{-\gamma \|X[i] - X[j]\|^2}$, and $SK = \mathbf{y}^T \cdot \mathbf{y} \cdot K$.
 - 3 **for** $t = 1$ **to** $iter$ **do**
 - 4 Choose $A_t \in [m]$, where $|A_t| = batch$, uniformly at random. Set $SK_{sub} \in \mathbb{R}^{batch \times m}$ where $SK_{sub}[i] = SK[A_t[i]]$.
 - 5 Calculate $T = SK_{sub} \cdot \alpha$.
 - 6 Set $A_t^+ = \{A_t[i] : T[i] - \frac{batch \cdot t \cdot c}{m} < 0\}$ and $\Delta_\alpha \in \{0, 1\}^m$ where $\Delta_\alpha[i] = 1$ for $i \in A_t^+$ and 0 for $i \notin A_t^+$.
 - 7 Update $\alpha = \alpha + \Delta_\alpha$.
 - 8 **end**
-

IV. PROBLEM FORMULATION

A. System Model

Our setting supposes a two-party MIoT scenario without a trusted third-party. The two parties P_0 and P_1 are likely to be hospitals or research institutes, and they both have the data of their patients. The data includes private information of the patients (e.g., biometric identifiers, allergies) and should not

be revealed to other parties. We formulate the private dataset of P_0 and P_1 as $X_0 \in \mathbb{R}^{m \times n}$ and $X_1 \in \mathbb{R}^{m \times n}$ respectively, and the private labels of the corresponding dataset as $\mathbf{y}_0 \in \mathbb{R}^m$ and $\mathbf{y}_1 \in \mathbb{R}^m$, where m is the size of the dataset and n is the number of features¹.

P_0 and P_1 now wish to jointly train an RBF kernel SVM model by their private dataset. The process includes the preparation phase and the training phase, and we ensure **not a bit** of information related to the original data of P_0 and P_1 is revealed to the other party during the process.

The goal of the preparation phase is to compute the RBF kernel matrix securely. Given K in Equation 1 generated by X_0 and X_1 , and $SK = (\mathbf{y}_0, \mathbf{y}_1)^T \cdot (\mathbf{y}_0, \mathbf{y}_1) \cdot K$, we assume that the row and column index of K and SK in $[0, m)$ corresponds to the index of data in X_0 , and the index in $[m, 2m)$ corresponds to the index of data in X_1 . We then split SK into 4 sub-matrices, SK_{00} , SK_{01} , SK_{10} and $SK_{11} \in \mathbb{R}^{m \times m}$ which respectively corresponds to the upper-left, upper-right, lower-left and lower-right part of SK . While SK_{00} and SK_{11} can respectively be computed by P_0 and P_1 locally, we aim to design a privacy-preserving protocol to securely evaluate SK_{01} and SK_{10} , and finally let P_1 retrieve the encrypted SK .

The goal of the training phase is to let P_i learn α_i where α_i is the sub-vector of α that corresponds to the index of data in X_i . The secure model inference is then achieved by P_0 and P_1 in an online manner. Note that sharing the whole model to either party for offline model inference should not be allowed since it carries information of X_0 and X_1 .

In terms of computation models, our scenario can be deployed in either the outsourcing mode (as in [29] and [24]) or the collocated mode where both parties jointly perform the computation. For ease of exposition, we focus on the outsourcing mode, in which P_1 conducts the main homomorphic operations on ciphertexts, while P_0 is responsible for generating the encryption keys and assisting P_1 in handling non-polynomial operations.

B. Threat Model

Our protocol considers security against a semi-honest adversary $\mathcal{A}$ running in probabilistic polynomial time, and it follows the simulation paradigm [45], [46], [47]. The security parameter is denoted as λ . The adversary $\mathcal{A}$ is computationally bounded and corrupts either P_0 or P_1 . Given the protocol $\Pi_{\mathcal{F}}$ that calculates the function $\mathcal{F}$, our security is modeled by defining two interactions: the real interaction where P_0 and P_1 execute $\Pi_{\mathcal{F}}$ in the presence of $\mathcal{A}$ and environment $\mathcal{Z}$, and the ideal interaction where a trusted third-party receives input from P_0 and P_1 , computes $\mathcal{F}$ and sends the respective output to the two parties. Our security requires that for every real adversary $\mathcal{A}$, there is an ideal adversary $\mathcal{S}$ such that no $\mathcal{Z}$ interacting with the adversaries can distinguish between real and ideal interactions. Our protocols involve invoking multiple

¹We assumed that the data volumes of P_0 and P_1 should be similar; otherwise, the party with significantly more data may prefer local training. If the quantity is not identical, the party that has less data can complement its data to the amount of data held by the other party by data augmentation or replicating existing data.

sub-protocols, and we describe them using hybrid model as in [21], [12]. A protocol invoking $\mathcal{F}$ is said to be in $\mathcal{F}$ -hybrid model.

V. SECURE RBF KERNEL FUNCTION EVALUATION

Before delving into the details of our RBF kernel function evaluation protocol, we introduce the *TruncSwitch* protocol in Algorithm 2 which extends from the one-bit approximate truncation protocol in [13]. The functionality $F_{TS}([a]^p, S, q)$ is to truncate $[a]^p$ by S and switch the modulus to q . The communication overhead for F_{TS} is free for $q = \frac{p}{S}$ and $2\lambda + \lceil \log q \rceil + 2$ bits otherwise. The security follows in the $((\frac{2}{1})\text{-OT}_1, F_{B2A})$ -hybrid.

Algorithm 2: $F_{TS}([a]^p, S, q)$.

Input: $[a]^p$ where $a \geq 0$ and $p = 2^k$, descaling factor $S = 2^n$, output modulus q

Output: $[\frac{a}{S}]^q$

```

1 if  $q = \frac{p}{S}$  then
2   | For  $b \in \{0, 1\}$ ,  $P_b$  outputs  $[\frac{a}{S}]_b^q = [a]_b^p - n$ .
3 end
4 else
5    $P_0$  generates a random bit  $[w]_0^2$ , and sets  $(s^0, s^1) = ([w]_0^2, [w]_0^2 \oplus MSB([a]_0^p))$ .
6    $P_0$  and  $P_1$  invoke  $(\frac{2}{1})\text{-OT}_1$ , where  $P_0$  is the sender with input  $(s^0, s^1)$  and  $P_1$  is the receiver with choice bit  $MSB([a]_1^p)$ .  $P_1$  learns  $[w]_1^2$  as output.
7    $P_0$  and  $P_1$  invoke  $F_{B2A}([w]^2)$  and obtain  $[w]^q$ .
8   For  $b \in \{0, 1\}$ ,  $P_b$  outputs  $[\frac{a}{S}]_b^q = ([a]_b^p - n) - [w]_b^q \cdot 2^{k-n} \mod q$ .
9 end

```

In the following subsections, we present NASE for secure exponentiation and our RBF kernel evaluation protocol.

A. NASE: Neat and Accurate Secure Exponentiation

We aim to calculate the exponential function of a **non-positive** exponent which is the case of RBF kernel function evaluation. It should be noted that the exact computation of the exponential function is not feasible even in plaintext, since the base e is a transcendental number and computers have limited precision to represent it. Therefore, We turn to computing an **approximation** of the exponential function.

To achieve secure exponentiation, several parameters are needed: the output scaling factor S , the secret sharing of the non-positive exponent $[g]^q$ scaled by S_0 , the minimum value of g as g_{min} and an output modulus p . For the RBF kernel function evaluation task, the estimation of g_{min} can either be acquired as $-S_0\gamma n$ for 0-1 normalized data or inferred by prior knowledge on the data. The modulus q can be either a prime number or a power-of-2 integer for NASE. We assume that it is a power-of-2 integer here, and it should satisfy $q \geq -2S_0g_{min}$ so that $[g]_0^q + [g]_1^q = g \mod q$. The value p should be set to be bigger than the maximum of $Se^{\bar{g}}$ where $\bar{g} = \frac{g}{S_0}$.

Given the inputs mentioned above, the goal is then to compute an approximation of $[Se^{\bar{g}}]^p$. When the two shares of

$[g]^q$ do not wrap around the modulus q , $e^{\bar{g}}$ is equal to $e^{\bar{g}_0} \cdot e^{\bar{g}_1}$ where $\bar{g}_i = \frac{[g]_i^q}{S_0}$. Another case is when the two shares wrap around q . There are two possibilities in such case, namely the sum of two shares gets smaller than $-\frac{q}{2}$ or greater than $\frac{q}{2} - 1$. The former situation will not be considered in our context since it requires the term being shared to be a positive number, while g is non-positive. As for the latter circumstance, the exponential function is thus equal to $e^{\bar{g}_0} \cdot e^{\bar{g}_1} \cdot e^{-\bar{q}}$ where $\bar{q} = \frac{q}{S_0}$. Therefore, the following equation holds:

$$e^{\bar{g}} = (1 - wrap) \cdot (e^{\bar{g}_0} \cdot e^{\bar{g}_1}) + wrap \cdot (e^{\bar{g}_0} \cdot e^{\bar{g}_1} \cdot e^{-\bar{q}}), \quad (2)$$

where $wrap = \mathbf{1}\{[g]_0^q + [g]_1^q = g + q\}$. Further rescaling the terms in the equation by $S_1^2 \lceil e^{\bar{q}} \rceil$ where $S_1 \in Z$ is a public scaling factor and discretizing them for MPC-based fixed-point computation, we then have:

$$S_1^2 \lceil e^{\bar{q}} \rceil e^{\bar{g}} \approx (1 - wrap) \cdot (\lfloor S_1 e^{\bar{g}_0} \rfloor \cdot \lfloor S_1 e^{\bar{g}_1} \rfloor \cdot \lceil e^{\bar{q}} \rceil) + wrap \cdot (\lfloor S_1 e^{\bar{g}_0} \rfloor \cdot \lfloor S_1 e^{\bar{g}_1} \rfloor). \quad (3)$$

The term $\lfloor S_1 e^{\bar{g}_i} \rfloor$ can be computed locally by P_i , and $\lfloor S_1 e^{\bar{g}_0} \rfloor \cdot \lfloor S_1 e^{\bar{g}_1} \rfloor$ can be computed securely by a single F_{CT} protocol. The secure computation of $wrap$ can be done using F_{Mill} , but it brings heavy communication and computation overhead. We show that the wrap can be identified in a more efficient way based on the following lemma (proof shown in the supplementary material).

Lemma 1: Given $[g]^q$ where $g \leq 0$ and $wrap = \mathbf{1}\{[g]_0^q + [g]_1^q = g + q\}$, $wrap = 1$ when $\mathbf{1}\{[g]_0^q > 0\} \wedge \mathbf{1}\{[g]_1^q > 0\} = 1$.

We use this lemma to turn the wrap checking task into checking the most-significant bit of each share which can be done without interactions between parties. Applying a single $(\frac{2}{1})\text{-OT}_1$ to the most-significant bit information, we can obtain the boolean secret sharing of $wrap$ with only $\lambda + 2$ bits of communication.

The workflow of NASE is illustrated in Algorithm 3. Lines 1-2 are to compute the secret sharing of $\lfloor S_1 e^{\bar{g}_0} \rfloor \cdot \lfloor S_1 e^{\bar{g}_1} \rfloor$. Lines 3-4 produce $\langle wrap \rangle$ based on Lemma 1, and line 5 is to calculate the secret sharing of $wrap \cdot \lfloor S_1 e^{\bar{g}_0} \rfloor \cdot \lceil e^{\bar{q}} \rceil$. Line 6 is to calculate the secret sharing of the scaled exponential function. The sharing is finally truncated by 2^{s_2} and switched modulus to p for later applications in line 7.

The security of NASE follows in the $(F_{CT}, (\frac{2}{1})\text{-OT}_1, F_{MUX}, F_{TS})$ -hybrid, and its total communication cost is $(l_e + 5)\lambda + \frac{3}{2}l_e^2 + \frac{9}{2}l_e + s_2 + 4$ bits. We compare NASE with SIRNN and the protocol in [22] in terms of the number of OT invocations and overall communication cost. [22] does not explicitly specify how their only interactive MTA protocol is implemented, and we assume it is achieved via OT as in our work. The protocol in [22] requires more than twice the number of OT calls as NASE, and its communication overhead is similarly more than doubled. This is primarily because their protocol operates over a large ring to mitigate the risk of computational failure, which significantly increases the cost of their only interactive sub-protocol (MTA). Compared to SIRNN, NASE is still much more lightweight concerning the fact that $\log g_{min}$ is typically smaller than s_1 .

Algorithm 3: $F_{NASE}(p, S_0, S_1, s_2, [g]^q)$.

Input: output ring p , input scaling factor S_0 , intermediate scaling factor S_1 , output truncation bit $s_2, [g]^q$

Output: $[\frac{S_1^2}{2^{s_2}} \lfloor e^{\frac{g}{S_0}} \rfloor \cdot e^{\frac{g}{S_0}} \rfloor^p$

- 1 For $i \in \{0, 1\}$, P_i computes $E_i = \lfloor S_1 \cdot e^{\bar{g}_i} \rfloor$.
- 2 P_0 and P_1 invoke $F_{CT}(E_0, E_1, l_e)$ and output $[w]^{4^{l_e}}$, where $l_e = \lceil \log S_1 \rceil + \lceil \log e^{\frac{g}{2}} \rceil + 1$.
- 3 P_0 generates a random bit r_0 and sets $\langle wrap \rangle_0 = r_0$.
- 4 P_0 and P_1 invoke $\binom{2}{1}$ -OT₁, where P_0 is the sender with input $(r_0, r_0 \oplus 1 \oplus MSB([g]_0^q))$, and P_1 is the receiver with choice bit $1 \oplus MSB([g]_1^q)$. P_1 learns the output as $\langle wrap \rangle_1$.
- 5 P_0 and P_1 invoke $F_{MUX}(\langle wrap \rangle, [w]^{4^{l_e}})$ and output $[v]^{4^{l_e}}$.
- 6 For $i \in \{0, 1\}$, P_i locally computes $[S_1^2 \lfloor e^{\bar{g}} \rfloor \cdot e^{\bar{g}}]_i^{4^{l_e}} = [v]_i^{4^{l_e}} + \lfloor e^{\frac{g}{S_0}} \rfloor \cdot ([w]_i^{4^{l_e}} - [v]_i^{4^{l_e}}) \bmod 4^{l_e}$.
- 7 P_0 and P_1 invoke $[\frac{S_1^2}{2^{s_2}} \lfloor e^{\bar{g}} \rfloor \cdot e^{\bar{g}}]^p = F_{TS}([S_1^2 \lfloor e^{\bar{g}} \rfloor \cdot e^{\bar{g}}]^{4^{l_e}}, s_2, p)$ and output it.

TABLE I: The comparison of communications in SIRNN, [22] and NASE, where $s_1 = \log S_1$ and $l_e = s_1 + \lceil \log g_{min} \rceil + 1$. The parameter for SIRNN and [22] follows their default setting.

Protocol	No. of OTs	Comm. (bits)
SIRNN	$23 + 2s_1 + s_2$	$(6s_1 + 25)\lambda + 3s_1 + 51s_1 + 8392$
[22]	$2l_e + 38$	$(2l_e + 38)\lambda + 6l_e^2 + 153l_e + 2185$
NASE	$l_e + 5$	$(l_e + 5)\lambda + \frac{3}{2}l_e^2 + \frac{9}{2}l_e + s_2 + 4$

The upper bound of the error between $\frac{S_1^2}{2^{s_2}} \lfloor e^{\bar{g}} \rfloor \cdot e^{\bar{g}}$ and the output of NASE, err_{max} , is the rounding error produced by Equation 3 plus the bit truncation error and is formulated as:

$$err_{max} = \max \left(\frac{e^{\frac{g}{2}} + 1}{2S_1}, \frac{1}{2S_1} + \left| 1 - \frac{\lfloor e^{\bar{g}} \rfloor}{e^{\bar{g}}} \right| \right) + \frac{2^{s_2} - 1}{S_1^2 \cdot e^{-\frac{g}{2}} \cdot \lfloor e^{\bar{g}} \rfloor}. \quad (4)$$

We prove the correctness of Equation 4 in the supplementary material, and it provides a sufficient but not necessary condition for NASE to achieve secure exponentiation in a desired precision by setting S_1, s_2, q and S_0 accordingly. This condition can usually be relaxed in practice.

B. Privacy-preserving RBF Kernel Function Computation

Concerning the input data $\mathbf{x}_0$ from P_0 and $\mathbf{x}_1$ from P_1 for secure RBF kernel function computation, the first step is to obtain the exponent $[g]^q = [-S_0 \gamma ||\mathbf{x}_0 - \mathbf{x}_1||^2]^q$ given $q \geq -2g_{min}$. The term g can be computed by the equation $g = 2S_0 \gamma \mathbf{x}_0 \cdot \mathbf{x}_1 - S_0 \gamma (||\mathbf{x}_0||^2 + ||\mathbf{x}_1||^2)$. While $\gamma ||\mathbf{x}_i||^2$ can be locally computed by P_i , $\gamma \mathbf{x}_0 \cdot \mathbf{x}_1$ can be evaluated using the F_{MCT} protocol. After obtaining the secret sharing of the exponent, NASE can be applied to securely evaluate the kernel function. We show the secure RBF kernel function evaluation protocol, F_{RBF} , in Algorithm 4. The security follows in the $(F_{MCT}, F_{TS}, F_{NASE})$ -hybrid.

Algorithm 4: $F_{RBF}(q, \gamma, \mathbf{x}_0, \mathbf{x}_1, S_0, p, S_1, s_2)$.

Input: $q, \gamma, \mathbf{x}_0$ and $\mathbf{x}_1 \in \mathbb{R}^n, S_0, p, S_1, s_2$

Output: $[\frac{S_1^2}{2^{s_2}} \lfloor e^{\bar{g}} \rfloor e^{-\gamma ||\mathbf{x}_0 - \mathbf{x}_1||^2}]^p$

- 1 P_0 and P_1 invoke $F_{MCT}(\lfloor S_0 \mathbf{x}_0 \rfloor, \lfloor S_0 \gamma \mathbf{x}_1 \rfloor, S_0, q)$ to output $[S_0^2 \gamma \mathbf{x}_0 \mathbf{x}_1]^{S_0 q}$.
- 2 P_0 and P_1 invoke $F_{TS}([S_0^2 \gamma \mathbf{x}_0 \mathbf{x}_1]^{S_0 q}, S_0, q)$ and output $[S_0 \gamma \mathbf{x}_0 \mathbf{x}_1]^q$.
- 3 For $b \in \{0, 1\}$, P_b calculates $[g]_b^q = 2[S_0 \gamma \mathbf{x}_0 \mathbf{x}_1]_b^q - \lfloor S_0 \gamma ||\mathbf{x}_b||^2 \rfloor \bmod q$.
- 4 P_0 and P_1 invoke $F_{NASE}(p, S_0, S_1, s_2, [g]^q)$ and output $[\frac{S_1^2}{2^{s_2}} \lfloor e^{\bar{g}} \rfloor e^{-\gamma ||\mathbf{x}_0 - \mathbf{x}_1||^2}]^p$.

VI. SECURE TWO-PARTY RBF SVM TRAINING

A. Protocol Overview

Inspired by the iterative framework of PEGASOS, the workflow of our secure RBF SVM training protocol is shown in Figure 1. We use BFV to achieve secure dot product computation and MPC technique for secure comparisons. The conversion method between BFV ciphertexts stored at P_1 and secret sharings between P_0 and P_1 over the same prime field $\mathbb{Z}_p$ is provided in Algorithm 5 and 6, and their security follows [12].

Algorithm 5: $F_{A2C}([\mathbf{x}]^p)$

Input: $[\mathbf{x}]^p$

Output: $enc(\mathbf{x})$ to P_1

- 1 P_0 encrypts $[\mathbf{x}]_0^p$ into BFV ciphertext and sends it to P_1 .
- 2 P_1 outputs $enc([\mathbf{x}]_0^p) \boxplus [\mathbf{x}]_1^p$.

Algorithm 6: $F_{C2A}(enc(\mathbf{x}))$

Input: $enc(\mathbf{x})$ from P_1

Output: $[\mathbf{x}]^p$

- 1 P_1 samples a random vector $\mathbf{r} \in \mathbb{Z}_N^p$ uniformly, where p and N is respectively the plaintext modulus and polynomial degree of BFV ciphertext.
- 2 P_1 sends $enc(\mathbf{x}) \boxplus -\mathbf{r}$ to P_0 . P_0 decrypts it as output, and P_1 sets $\mathbf{r}$ as output.

In the preparation phase of our training protocol, we let P_i compute SK_{ii} locally. We then securely evaluate $[SK_{01}]^p$ by invoking the F_{RBF} protocol together with one F_{MUX} protocol. Additionally, $[SK_{10}]^p$ can be obtained by the transpose of $[SK_{01}]^p$, and we achieve it by making P_i transpose $[SK_{01}]_i^p$. The two parties then pack their shares and their sub-matrix of SK to obtain $G_{00}, [G_{01}]^p, [G_{10}]^p$ and G_{11} for later computation in the training phase. The computation party P_1 obtains $enc(G_{01})$ and $enc(G_{10})$ by respectively invoking $F_{A2C}([G_{01}]^p)$ and $F_{A2C}([G_{10}]^p)$ with P_0 . The party P_0 finally encrypts G_{00} as $enc(G_{00})$ and sends it to P_1 so that P_1 forms the packed and partially encrypted $SK, \hat{G}$.

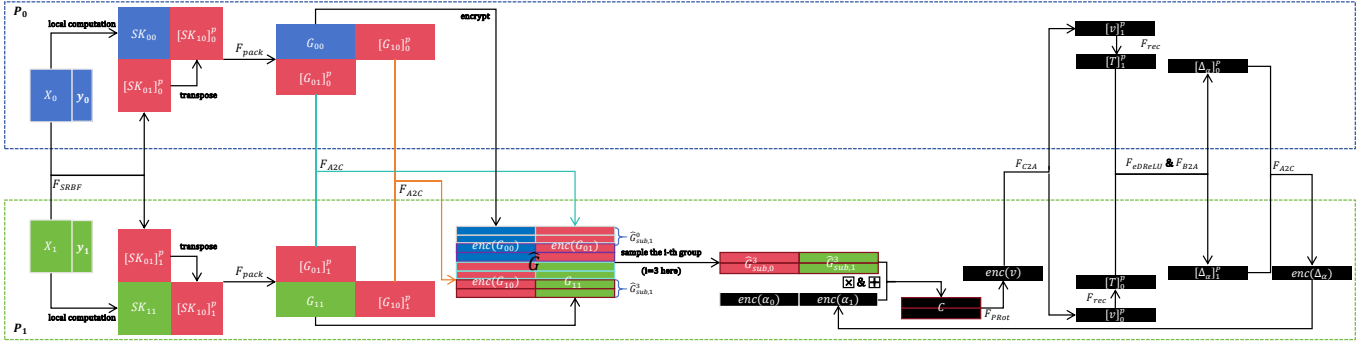


Fig. 1: An illustration on the workflow of our training protocol. All terms are assumed to be scaled.

Concerning each iteration in the training phase of our protocol, the first step is to sample SK_{sub} from SK . Achieving the mini-batch sampling as in PEGASOS is usually difficult for BFV ciphertexts, and we introduce a so-called group-batch sampling which can be easily achieved by selecting a subset of $\hat{G}$. The detail of group-batch sampling will be discussed in section VI-B. We then compute the dot product $T = SK_{sub} \cdot \alpha$ by homomorphic multiplication, addition and rotation. To optimize the secure dot product computation, we design the F_{PRot} and F_{rec} functionalities, and we delve into their detail in section VI-C. The result of the dot product is converted to the secret sharing by F_{C2A} , and the F_{eDReLU} protocol (discussed in section VI-D) is applied to securely obtain the bit secret sharing of the parameter update, $\Delta_\alpha = \mathbf{1}\{T < \frac{batch \cdot t \cdot c}{m}\}$. The update is finally switched modulus by F_{B2A} and used to update $enc(\alpha_0)$ or $enc(\alpha_1)$.

In the following subsections, we introduce our novel designs to improve the efficiency of secure SVM training:

- The modification of mini-batch sampling algorithm which enables both easy extraction of encrypted SK_{sub} from $\hat{G}$ in each iteration and more efficient packing strategy for the signed kernel matrix.
- The optimization on secure dot product computation. Based on the fact that the F_{C2A} protocol uses a ciphertext of N slots to transmit $batch$ terms, we take advantage of the redundancy and reduce the number of ciphertext rotations by several times.
- A new comparison protocol, F_{eDReLU} that enjoys less communication cost.

Finally, we show how to achieve RBF kernel SVM inference securely by our trained model.

B. Group-Batch Sampling for Efficient Data Packing

Mini-batch PEGASOS samples $batch$ rows of SK to form SK_{sub} in each iteration. If we apply mini-batch sampling to our training protocol directly, the method to achieve such sampling for $\hat{G}$ is related to the packing strategy of SK_{00} , $[SK_{01}]^p$, $[SK_{10}]^p$ and SK_{11} to G_{00} , $[G_{01}]^p$, $[G_{10}]^p$ and G_{11} . A native approach is to encode each line of the sub-matrices SK_{ij} individually and encrypt it into one ciphertext for $m \leq N$ or multiple ciphertexts for $m > N$. The sampling essentially becomes choosing the corresponding ciphertexts or plaintexts, but such naive encoding introduces redundancy in

computation and communication when $m\%N \neq 0$. Besides, it eliminates the possibility of packing multiple lines of the kernel matrix into one ciphertext densely and processing them in SIMD manner for $m \leq N$, and the chance to apply more efficient matrix encoding strategies for secure dot product computation like Halevi-Shoup packing [48] for $m > N$. Applying dense packing or Halevi-Shoup packing directly to SK_{ij} is neither feasible since sampling SK_{sub} from $\hat{G}$ packed in such manner would be difficult and impractical.

To resolve the trade-off between sampling and efficiency, we alter the mini-batch sampling strategy and introduce group-batch sampling. Group-batch sampling splits the index of dataset into $\lceil \frac{m}{batch} \rceil$ fixed subsets of size $batch$, $\{A^0, A^1, \dots, A^{\lceil \frac{m}{batch} \rceil - 1}\}$, and defines a fixed set of sub-matrices $SK_{sub} = \{SK_{sub}^0, SK_{sub}^1, \dots, SK_{sub}^{\lceil \frac{m}{batch} \rceil - 1}\}$ where SK_{sub}^i corresponds to A^i rows of SK . In each iteration, we randomly choose a sub-matrix SK_{sub}^i from SK_{sub} instead of sampling $batch$ rows from SK . The compact packing and encoding-based optimizations can then be applied to those fixed sub-matrices in SK_{sub} individually. Algorithm 7 illustrates the workflow of group-batch sampling implemented within the PEGASOS framework.

We now discuss the packing strategy for each SK_{sub}^i or $[SK_{sub}^i]_b^p$. To securely calculate T in each iteration, we split it into $SK_{sub,0}^i \cdot \alpha_0 + SK_{sub,1}^i \cdot \alpha_1$ for computation. The symbol $SK_{sub,j}^i$ indicates the sub-matrix of SK_{sub}^i that overlaps with SK_{0j} or SK_{1j} . Since SK_{11} is plaintext to P_1 , and SK_{00} , SK_{01} and SK_{10} together with α are in ciphertext, such partition allows us to pack those $SK_{sub,1}^i$ that are sub-matrices of SK_{11} in plaintext, and the dot product computation for them can be carried between plaintext and ciphertext. The goal is then to compactly and efficiently pack each $SK_{sub,j}^i$ into $G_{sub,j}^i$ or $[SK_{sub,j}^i]_b^p$ into $[G_{sub,j}^i]_b^p$. We introduce our packing function, F_{pack} , in Algorithm 8. For $m > N$, we apply Halevi-Shoup packing. As for the case when $m \leq N$, we densely pack each sub-matrix by packing multiple rows into each ciphertext. By such packing methods, we ensure a compact data structure which minimizes the communication overhead for one-time setup and facilitates efficient secure dot product computation in the subsequent step at the same time.

Algorithm 7: Group-batch PEGASOS.**Input:** $X \in \mathbb{R}^{m \times n}$, $\mathbf{y}$, γ , c , $iter$, $batch$ **Output:** α

```

1 Initialize  $\alpha \in \{0\}^m$ .
2 Compute RBF kernel matrix  $K \in \mathbb{R}^{m \times m}$  where
    $K(i, j) = e^{-\gamma \|X[i] - X[j]\|^2}$ , and  $SK = \mathbf{y}^T \cdot \mathbf{y} \cdot K$ .
3 For  $A = \{0, 1, \dots, m-1\}$ , split  $A$  into  $\lceil \frac{m}{batch} \rceil$  groups
   where the  $(i+1)$ -th group
    $A^i = \{ibatch, ibatch+1, \dots, (i+1)batch-1\}$  for
    $i \neq \lceil \frac{m}{batch} \rceil$  and  $A^i = \{ibatch, ibatch+1, \dots, m-1\}$ 
   for  $i = \lceil \frac{m}{batch} \rceil$ .
4 For  $i \in \{0, 1, \dots, \lceil \frac{m}{batch} \rceil\}$ , set  $SK_{sub}^i \in \mathbb{R}^{batch \times m}$ 
   where  $SK_{sub}^i[j] = SK[A^i[j]]$ .
5 for  $t = 1$  to  $iter$  do
6   choose  $i$  from  $\{0, 1, \dots, \lceil \frac{m}{batch} \rceil\}$  uniformly at
   random.
7   Calculate  $T = SK_{sub}^i \cdot \alpha$ .
8   Set  $A_t^+ = \{A^i[j] : T[j] - \frac{batch \cdot t \cdot c}{m} < 0\}$  and
    $\Delta_\alpha \in \{0, 1\}^m$  where  $\Delta_\alpha[j] = 1$  for  $j \in A_t^+$  and
   0 for  $j \notin A_t^+$ .
9   Update  $\alpha = \alpha + \Delta_\alpha$ .
10 end

```

C. Optimized Dot Product Computation via Partial Rotation

We discuss how to reduce the number of ciphertext rotation during computation of $SK_{sub,j}^i \cdot \alpha_j$ in each iteration. Assume that $\hat{G}_{sub,j}^i = G_{sub,j}^i$ for $G_{sub,j}^i \in G_{11}$ or $enc(G_{sub,j}^i)$ otherwise, and $\hat{G}_{sub,j}^i$ has $c_g = \frac{batch \cdot 2^{\lceil \log m \rceil}}{N}$ ciphertexts or plaintexts. The parameters α_0 and α_1 are replicated row-wise to be a vector of length N and encrypted into a single ciphertext for $m \leq N$ or multiple ciphertexts for $m > N$ as $enc(\alpha_0)$ and $enc(\alpha_1)$. To compute T using $enc(\alpha_0)$, $enc(\alpha_1)$ and $\hat{G}_{sub,j}^i$, each ciphertext or plaintext in $\hat{G}_{sub,j}^i$ requires one homomorphic multiplication, $\lceil \log m \rceil$ rotations and one element extraction for $m \leq N$. Finally, a summation is required to merge all the intermediate ciphertexts into one ciphertext. For $m > N$, one homomorphic multiplication and one rotation are needed per ciphertext or plaintext. Then, the intermediate ciphertexts are summed together to be rotated $\lceil \log \frac{N}{batch} \rceil$ times. Since the computation cost of one ciphertext rotation operation is similar to that of one homomorphic multiplication operation, it is necessary to minimize the rotation times for both packing strategies.

It should be noted that the dot product result will be subtracted from a public number $\frac{batch \cdot t \cdot c}{m}$ and compared with 0 to obtain $[\Delta_\alpha]^2$ in our protocol. Since we utilize MPC technique to achieve secure comparison, we need to transform the ciphertext result into secret sharing by F_{C2A} . While ensuring that only a minimal amount of $\lceil \frac{batch}{N} \rceil$ (1 in our case since $batch \ll N$) ciphertext is sent to P_0 during the F_{C2A} protocol, the process is essentially sending $batch$ terms by a ciphertext which is capable of carrying N numbers. Thus, we utilize the slot redundancy and introduce F_{PRot} and F_{rec} as shown in Algorithm 9 and 10. Given the intermediate computation results C where $C[z] = \hat{G}_{sub,0}^i[z] \boxtimes enc(\alpha_0) \boxplus \hat{G}_{sub,1}^i[z] \boxtimes enc(\alpha_1)$, the

Algorithm 8: $F_{pack}(SK_{ij}$ or $[SK_{ij}]_b^p, N, batch)$.**Input:** SK_{ij} or $[SK_{ij}]_b^p$, N , $batch$ **Output:** G_{ij} or $[G_{ij}]_b^p$

```

1 Set  $U = SK_{ij}$  or  $[SK_{ij}]_b^p$ ,  $s = 2^{\lceil \log m \rceil}$ ,
    $k = i \cdot \lceil \frac{m}{batch} \rceil$  and  $u = \frac{N}{s}$ .
2 if  $m > N$  then
3   for  $z = 0$  to  $\lceil \frac{m}{batch} \rceil - 1$  do
4     Apply Halevi-Shoup packing on
        $U[batch \cdot z : batch \cdot (z+1)]$  to obtain  $G_{sub,j}^{k+z}$ 
       or  $[G_{sub,j}^{k+z}]_b^p \in \mathbb{Z}_{\frac{batch}{u}}^p \times N$ .
5   end
6 end
7 else
8   for  $z = 0$  to  $\lceil \frac{m}{batch} \rceil - 1$  do
9     Initialize  $\hat{G}_{sub,j}^{k+z}$  or  $[\hat{G}_{sub,j}^{k+z}]_b^p \in \mathbb{Z}_{\frac{batch}{u}}^p \times N$ .
10    for  $t = 0$  to  $\frac{batch}{u} - 1$  do
11      Set  $\hat{G}_{sub,j}^{k+z}[t]$  or  $[\hat{G}_{sub,j}^{k+z}]_b^p[t] = \mathbf{g}_t \in \mathbb{Z}^N$ 
        where  $\mathbf{g}_t[a] = U[u \cdot t + z \cdot batch + \lfloor \frac{a}{s} \rfloor][a \bmod s]$ 
        for  $(a \bmod s) < m$  and
         $\mathbf{g}_t[a] = 0$  for  $(a \bmod s) \geq m$ .
12    end
13  end
14 end
15 Output  $G_{ij} = \{G_{sub,j}^k, G_{sub,j}^{k+1}, G_{sub,j}^{k+2}, \dots\}$  or  $[G_{ij}]_b^p =$ 
    $\{[G_{sub,j}^k]_b^p, [G_{sub,j}^{k+1}]_b^p, [G_{sub,j}^{k+2}]_b^p, \dots\}$ .

```

idea of F_{PRot} is to partially rotate those ciphertexts in C for $m \leq N$ or simply skip rotation for $m > N$. The ciphertexts are then summed to one ciphertext, and F_{C2A} is invoked to obtain its secret sharing. Finally, P_0 and P_1 further rotate and sum these terms in plaintext by F_{rec} function to recover the secret sharing of the dot product. By such methodology, a large number of rotations in ciphertext by P_1 is transferred to rotations on plaintext which is almost free, and the process does not yield any additional overhead on communication. We compare the exact number of ciphertext rotations before and after applying our partial rotation strategy in Table II. Since $batch \ll N$ and $c_g \leq batch$ for dense packing, it can be concluded that our partial rotation strategy leads to noticeable computation savings for both packing methods.

TABLE II: Comparison of ciphertext rotations between the baseline dot-product computation protocol and ours.

Packing strategy	No. of ciphertext rotations	
	Baseline	Ours
Dense packing	$c_g \lceil \log m \rceil$	$c_g \lceil \log c_g \rceil$
Halevi-Shoup	$\log \frac{N}{batch} + c_g - 1$	$c_g - 1$

D. More Efficient Comparison Protocol

For the secure comparison protocol to compare $T[j] - \frac{batch \cdot t \cdot c}{m}$ with 0, prior work [12] provides the $DReLU$ protocol to solve it, and the communication cost given the secret sharing over $\mathbb{Z}_{2^l}$ is less than $\lambda l + 18l$ bits. When it comes to the secret sharing over a prime field $\mathbb{Z}_p$, the communication cost

Algorithm 9: $F_{PRot}(C, N, c_g, m)$.

Input: intermediate ciphertexts C, N, c_g, m
Output: $enc(\mathbf{v})$

```

1 Initialize the ciphertext  $enc(\mathbf{v})$  of a zero vector.
2 if  $m > N$  then
3   | Compute  $enc(\mathbf{v}) = \sum_{i=1}^{c_g} C[i]$ .
4 end
5 if  $m \leq N$  then
6   | for  $z = 0$  to  $c_g - 1$  do
7     | Initialize  $O = C[z]$ .
8     | for  $j = 0$  to  $\log c_g$  do
9       | Compute  $O = O \boxplus rot(O, 2^j)$ .
10    | end
11    | Compute  $Q = O \boxtimes M$ , where  $M$  is a mask
      | vector of length  $N$  that satisfies  $M[j] = 1$  for
      |  $j \bmod c_g = 0$  and  $M[j] = 0$  elsewhere.
12    | Compute  $enc(\mathbf{v}) = enc(\mathbf{v}) \boxplus rot(Q, z)$ .
13  | end
14 end

```

Algorithm 10: $F_{rec}([v]_b^p, m, c_g, batch, N)$.

Input: Secret share $[v]_b^p, m, c_g, batch, N$
Output: $[T]_b^p \in Z_p^{batch}$

```

1 if  $m > N$  then
2   | for  $i = 0$  to  $batch - 1$  do
3     | Calculate  $[T]_b^p[i] = \sum_{j=0}^{N/batch-1} [v]_b^p[i + j \cdot batch]$ .
4   | end
5 end
6 if  $m \leq N$  then
7   | for  $i = 0$  to  $c_g - 1$  do
8     | for  $j = 0$  to  $\frac{N}{2^{\lceil \log m \rceil}} - 1$  do
9       | Calculate
          |  $[T]_b^p[i \cdot \frac{N}{2^{\lceil \log m \rceil}} + j] = \sum_{z=0}^{\frac{2^{\lceil \log m \rceil} - 1}{c_g}} [v]_b^p$ 
          |  $[2^{\lceil \log m \rceil} \cdot j + c_g \cdot z + i]$ .
10    | end
11  | end
12 end

```

becomes $\frac{3}{2}\lambda(\eta + 1) + 31\eta$ where $\eta = \lceil \log p \rceil$. It can be observed that the communication burden is much heavier for $DReLU$ protocol over the prime field, while the field in our scenario should be prime since the plaintext modulus of the BFV ciphertext is a prime number. In this paper, we propose an error-tolerant solution, F_{eDReLU} , and the workflow is shown in Algorithm 11. When p is a power-of-2 integer, F_{eDReLU} is exactly the $DReLU$ protocol. When p is a prime, F_{eDReLU} still enjoys the same communication cost as $DReLU$ over $\mathbb{Z}_{2^l}$ while functioning similarly to $DReLU$.

Lemma 2: Given the input secret sharing $[x]^p$ where p is a prime number and $x \neq \lfloor \frac{p}{2} \rfloor$, the output $\langle \mathbf{1}\{x \geq 0\} \rangle_0$ and $\langle \mathbf{1}\{x \geq 0\} \rangle_1$ of $F_{eDReLU}([x]^p, l, s)$ satisfies $\langle \mathbf{1}\{x \geq 0\} \rangle_0 \oplus \langle \mathbf{1}\{x \geq 0\} \rangle_1 = \mathbf{1}\{x \geq 0\}$.

The proof of Lemma 2 is in the supplementary material.

Algorithm 11: $F_{eDReLU}([x]^p)$.

Input: $[x]^p$
Output: $\langle \mathbf{1}\{x \geq 0\} \rangle$

```

1 For  $b \in \{0, 1\}$ ,  $P_b$  calculates  $a_b = MSB_b \cdot \lceil \frac{p}{2} \rceil + [x]_b^p$ 
   where  $MSB_b$  is the most-significant bit of  $[x]_b^p$ .
2  $P_0$  and  $P_1$  invoke  $F_{Mill}(a_0, \lfloor \frac{p}{2} \rfloor - a_1)$  to learn
    $\langle carry \rangle$ .
3 For  $b \in \{0, 1\}$ ,  $P_b$  outputs  $\langle \mathbf{1}\{x \geq 0\} \rangle_b = MSB_b \oplus$ 
    $\langle carry \rangle_b \oplus b$ .

```

According to the lemma, F_{eDReLU} may not produce the correct output when $x = \lfloor \frac{p}{2} \rfloor$, but we can easily avoid the failure by setting p slightly larger than the range of x so that $x < \lfloor \frac{p}{2} \rfloor$. With our F_{eDReLU} , we reduce the communication cost for one secure comparison operation over the prime field from $\frac{3}{2}\lambda(\eta + 1) + 31\eta$ to less than $\lambda\eta + 18\eta$. The security of F_{eDReLU} follows trivially in the F_{Mill} -hybrid.

E. Secure SVM Inference

Our proposed training protocol outputs $enc(\alpha)$, and the next step is to let P_i retrieve α_i . Our secure approach is to recover α_0 for P_0 by transmitting the corresponding part of $enc(\alpha)$ directly from P_1 to P_0 for decryption. The α_1 can be securely revealed to P_1 by invoking F_{C2A} to the corresponding part of $enc(\alpha)$ and letting P_0 send $[\alpha_1]_0^p$ to P_1 for recovery.

We present the workflow of the secure inference of our trained SVM model in Algorithm 12. For an input data $\hat{x}$ from P_b to be evaluated, the algorithm outputs the predicted label to P_b . Assume that the index set of support vectors is SV , and there are z support vectors in total. SV_b indicates the index set of support vectors held by P_b , and its size is z_b . The security of the protocol follows in the F_{RBF} - F_{MUX} - F_{A2C} - F_{eDReLU} -hybrid.

VII. PERFORMANCE EVALUATION

We provide experimental results to show the effectiveness of NASE and our secure SVM training protocol. We use several real-world datasets from UCI repository and Kaggle, as listed in Table III, most of which correspond to typical MIoT application scenarios. The parameter n denotes the number of features in each dataset. The datasets are preprocessed by 0-1 normalization, and we remove data samples with missing value. An appropriate γ that has been tested in plaintext is assigned to each dataset. As for the hyper-parameters, we set $\lambda = 128$, $N = 8192$, $p = 281474976694273$, $c = 1$ and $q = \lceil -2S_0g_{min} \rceil$. Our experiments are conducted on Intel CPU i7-12700 @ 2.10 GHz. The implementation of NASE and our secure SVM training protocol is in python.

A. RBF Kernel Computation

To test the performance of NASE, we assume that the exponent input $[g]^q$ is already computed by P_0 and P_1 . The error metric is the mean relative error between the output of NASE and $\frac{S_1^2}{2^{s_2}} [e^{\frac{g}{s_0}}] e^g$. We compare NASE to SIRNN (the optimized

Algorithm 12: Secure kernel SVM inference.

Input: $SV_0, SV_1, \alpha_b, X_b$ and $\hat{\mathbf{x}}$ from P_b , α_{1-b} and X_{1-b} from P_{1-b} , S_0, S_1, s_2, q, p, N

Output: predicted label of $\hat{\mathbf{x}}$ to P_b

- 1 P_b computes $sum_b = \frac{S_1^2}{2^{s_2}} \lfloor e^{\frac{p}{S_0}} \rfloor \sum_{i=0}^{z_b-1} \mathbf{y}_b$
 $[SV_b[i]]_{\alpha_b} [SV_b[i]] K(X_b[SV_b[i]], \hat{\mathbf{x}})$.
- 2 For $i \in \{0, 1\}$, P_i initializes $[\mathbf{sk}]_b^p \in \mathbb{Z}_p^{len(SV_{1-b})}$.
- 3 **for** $k = 0$ to $z_{1-b} - 1$ **do**
- 4 P_b and P_{1-b} invoke
 $F_{RBF}(q, \gamma, \hat{\mathbf{x}}, X_{1-b}[SV_{1-b}[k]], S_0, p, S_1, s_2)$ and
 F_{MUX} to output $[\mathbf{sk}[k]]^p$.
- 5 **end**
- 6 For $i \in \{0, 1\}$, P_i repacks $[\mathbf{sk}]_b^p = F_{pack}([\mathbf{sk}]_b^p, N, 1)$.
- 7 P_b and P_{1-b} invokes $F_{A2C}([\mathbf{sk}]^p)$ and outputs
 $enc(\mathbf{sk})$ to P_{1-b} .
- 8 P_{1-b} computes $C = enc(\mathbf{sk}) \boxtimes \alpha_{1-b}$ and
 $\hat{U} = F_{P_{Rot}}(C, N, len(C))$.
- 9 P_{1-b} invokes $F_{C2A}(\hat{U})$ with P_b and outputs $[\hat{U}]^p$.
- 10 For $i \in \{0, 1\}$, P_i computes
 $[\sum_{i=0}^{z_{1-b}-1} \mathbf{sk}[i] \alpha_{1-b}^{SV_{1-b}[i]}]_i^p =$
 $F_{rec}([\hat{U}]_i^p, len(SV_b), len(C), 1, N)$.
- 11 P_b sets $[sum]_b^p = sum_b + [\sum_{i=0}^{z_{1-b}-1} \mathbf{sk}[i] \alpha_{1-b}^{SV_{1-b}[i]}]_b^p$
mod p , P_{1-b} sets
 $[sum]_{1-b}^p = [\sum_{i=0}^{z_{1-b}-1} \mathbf{sk}[i] \alpha_{1-b}^{SV_{1-b}[i]}]_{1-b}^p$.
- 12 P_b and P_{1-b} invoke $F_{eDReLU}([sum]^p)$ and output
 $\langle \mathbf{1}(sum < 0) \rangle$.
- 13 P_{1-b} sends $\langle \mathbf{1}(sum < 0) \rangle_{1-b}$ to P_b . The party P_b
recovers $\mathbf{1}(sum < 0)$ and output -1 if
 $\mathbf{1}(sum < 0) = 1$ or 1 otherwise.

TABLE III: Datasets used for evaluation.

Dataset	Size	n	Train/test ratio	γ
Mines vs. Rocks	160	60	4:1	0.6
Heart Disease	303	13	5.5:1	0.55
WBC	683	9	3:1	0.8
AIDS Study 175	1280	23	4:1	0.6
Spambase	2560	57	4:1	1.35
Anemia	5120	11	4:1	2.4

version for inputs with known MSB) and the protocol in [22] on our simulated dataset. The result is shown in Table IV. The simulated dataset contains 2×10^5 exponents that are additively shared, and their values are uniformly sampled within the specific range indicated in the first column of the table. The parameter corresponding to SIRNN and NASE is respectively the scaling factor for LUT and s_1 . The parameter of [22] is the fractional bits for the fixed-point ring, and its failure probability is set to be 2^{-40} as in their default configuration. While achieving similar accuracy, NASE improves up to $1.65\times$ in computation time and $3.97\times$ in communication compared to SIRNN when in the range $[-4, 0]$. Under the worst case when the range is set to $[-14, 0]$, NASE still achieves $1.55\times$ computation and $2.64\times$ communication improvement. Compared to [22], NASE achieves $2.22\times$ speedup and $3.88\times$ communication improvement on average. This improvement is primarily attributed to the fact that NASE requires significantly

fewer OT invocations for the F_{CT} sub-protocol compared to the MTA sub-protocol in [22]. The experimental result is in accordance to our theoretical analysis in Table I. Note that we do not conduct experiments on the input range whose lower bound is smaller than 14 since we found that $[-14, 0]$ is usually enough for RBF kernel applications during our experiments on real-world datasets. Besides, we can easily avoid using a larger range by setting a smaller γ .

We compare NASE with SIRNN and [22] on real-world dataset, and the result is shown in Table V. The exponents are $-\gamma||X_0 - X_1||^2$ and assumed to be pre-computed as additive secret sharings. While the data distribution of real-world datasets is more varied and unpredictable, NASE still outperforms SIRNN and [22] in efficiency.

To further evaluate the performance of NASE in secure RBF kernel evaluation task, we compare F_{RBF} to other privacy-preserving RBF kernel computation methods, namely Taylor extension [27] and RFF [20]. For fairness, we realize these two methods using F_{MCT} protocol, and the parameter for them is chosen so that their efficiency and accuracy are well-balanced. According to the comparison result in Table VI, it is observed that while Taylor extension is capable of handling datasets that produce exponents in a smaller range for RBF kernel evaluation and do not have too many features (e.g. WBC), its performance is catastrophic when the range and number of features get bigger. The same applies to RFF, and their kernel approximation errors both have a negative influence on the performance of subsequent SVM training. In contrast, our protocol is more invariant to exponent range and feature dimension, tested to be more robust and practical in real-world secure RBF kernel SVM applications.

B. Secure SVM Training

We run our secure SVM training protocol with different batch sizes and various datasets. The result is shown in Table VII. PT accuracy indicates the accuracy of the RBF kernel SVM model trained in plaintext. The runtime and communication overhead in the table exclude the cost in kernel computation phase. While the computation time tends to increase with the batch size getting bigger, the communication overhead becomes smaller. This is because the transmission of BFV ciphertexts constitutes the bulk of the communication during training, and the quantity of ciphertexts communicated is linear to iteration rounds. While attaining the accuracy of plaintext training, Our protocol achieves secure training on hundreds of data within seconds and on thousands of data within a few minutes, showing its computational efficiency. The communication cost is also kept low, which is in the order of megabytes for our datasets.

We compare our training protocol to [24] on UCI cardiotocography dataset. We set $iter = 200$ and $batch = 64$, and the one-time setup cost in the preparation phase of our protocol is taken into account. The comparison result² on computation time and accuracy is given in Figure 2. Concerning communication cost, we compare the communication overhead in Table

²The runtime and accuracy of [24] are directly cited from the experimental result in [24].

TABLE IV: The average runtime, communication cost and accuracy of one secure exponentiation on simulated dataset.

Range	Parameter			Runtime (μ s)			Comm. (bit)			Accuracy (%)		
	SIRNN	[22]	NASE	SIRNN	[22]	NASE	SIRNN	[22]	NASE	SIRNN	[22]	NASE
[-4,0]	14	14	14	36.33	49.27	21.98	16282	18788	4102	99.98	99.97	99.97
[-8,0]	19	19	19	44.05	60.19	27.28	22122	25745	6456	99.98	99.98	99.98
[-12,0]	23	23	24	50.49	69.89	32.07	26902	32745	9172	99.96	99.95	99.96
[-14,0]	25	26	28	55.05	79.21	35.44	29328	37510	11090	99.94	99.96	99.94

TABLE V: The comparison of secure exponentiation protocols on real-world dataset, given different network setting.

Dataset	Parameter			Runtime (s)			Comm. (MB)			Accuracy (%)		
	SIRNN	[22]	NASE	SIRNN	[22]	NASE	SIRNN	[22]	NASE	SIRNN	[22]	NASE
Mines vs. Rocks	19	18	24	0.37	0.57	0.26	10.80	13.81	4.48	99.99	99.98	99.98
Heart Disease	14	14	14	0.88	1.35	0.57	31.80	36.70	8.01	99.99	99.98	99.98
WBC	15	15	19	4.70	5.30	3.56	136.23	176.04	50.44	99.99	99.98	99.98
AIDS Study 175	15	15	18	14.38	22.09	10.76	544.94	704.16	194.59	99.97	99.96	99.96
Spambase	14	14	24	51.14	76.26	47.85	2035.25	3115.50	1146.50	99.98	99.97	99.96
Anemia	16	17	26	224.67	366.33	204.56	9300.00	14574.50	5263.50	99.96	99.98	99.96

TABLE VI: Comparisons of different kernel computation approaches. Kernel error indicates the mean relative error of kernel computation. Training accuracy is the accuracy of the SVM model trained and tested using corresponding kernel matrix.

Dataset	Runtime (s)			Comm. (MB)			Kernel error (%)			Training accuracy (%)		
	Taylor	RFF	Ours	Taylor	RFF	Ours	Taylor	RFF	Ours	Taylor	RFF	Ours
Mines vs. Rocks	124.68	0.19	0.35	13184.96	33.50	21.11	94.80	1003.60	0.02	65.00	64.38	75.31
Heart Disease	45.18	1.03	0.62	34337.67	268.55	20.27	4.45	17.84	0.02	83.77	73.93	85.32
WBC	0.94	7.76	3.83	210.36	2177.11	83.07	11.45	64.58	0.02	97.49	96.72	97.19
AIDS Study 175	134.10	52.54	12.15	3.50×10^6	17792.00	541.16	68.13	61.44	0.04	86.21	81.28	87.15
Spambase	326.18	445.24	56.04	1.22×10^6	73233.75	4655.50	0.05	22.11	0.04	91.39	85.63	91.33
Anemia	> 3000	2598.13	218.77	5.72×10^7	1.20×10^6	8305.00	6.13	39.86	0.04	97.66	95.80	98.18

TABLE VII: Performance of our secure SVM training protocol.

Dataset	Heart Disease		WBC		AIDS Study 175			Spambase			Anemia		
<i>batch</i>	64	128	64	128	32	64	128	32	64	128	32	64	128
<i>iter</i>	170	125	50	45	380	200	170	865	400	300	850	520	450
Runtime (s)	7.03	10.49	3.48	6.06	23.91	26.04	46.97	100.87	99.78	184.55	208.42	288.99	507.79
Comm. (MB)	118.86	97.47	34.96	35.09	250.38	139.83	132.56	569.93	279.67	233.93	560.05	363.57	350.89
Accuracy (%)	85.32	85.32	97.19	97.13	87.54	87.54	87.23	91.99	91.39	91.52	98.37	97.92	98.12
PT accuracy (%)	85.32	85.11	97.25	97.13	87.34	87.34	87.23	91.78	91.86	91.86	98.12	98.21	98.13

VIII. Our computation time and communication are an order of magnitude lower than [24]. This is due to the significant advantage of BFV-based polynomial evaluation over MPC, our optimization on ciphertext rotation times and the more efficient comparison protocol. Our accuracy is also around 4% higher than [24] on average due to the low kernel function computation error of NASE and the better strategy of learning rate selection in our training protocol over the fixed learning rate strategy in [24].

TABLE VIII: Amortized communication overhead (KB) to process an individual sample in the batch in each iteration.

m	250	500	1000
[24]	839.62	1669.09	3328.02
Ours	11.47	12.29	15.59

VIII. CONCLUSION

In this paper, we propose NASE, a neat and accurate secure exponentiation protocol for secure RBF kernel computation task. Our experiments show that NASE outperforms SIRNN and [22] in efficiency on our simulated and real-world dataset. We also propose a secure kernel SVM training protocol. By our algorithm, A secure training task on a dataset with several thousand records can be computed within minutes while ensuring a low communication cost.

As for the limitation of our work, it should be noted that the trained model is distributed at P_0 and P_1 , and the integral model cannot be shared to and recovered by either party since α_i carries information about X_i . Therefore, the absence of either party will make it impossible to achieve secure inference, and the inference has to be carried out jointly in an online manner. Our future work will focus on this issue, researching how to obtain a secure and offline kernel SVM

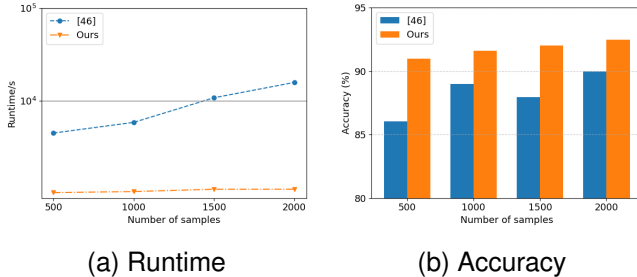


Fig. 2: The comparison between different secure SVM training protocols.

model for an individual party to make secure predictions without interactions.

REFERENCES

- [1] D. Valluru and I. J. S. Jeya, "IoT with cloud based lung cancer diagnosis model using optimal support vector machine," *Health Care Management Science*, vol. 23, no. 4, pp. 670–679, Dec. 2020. [Online]. Available: <https://doi.org/10.1007/s10729-019-09489-x>
- [2] G. Premalatha and V. T. Bai, "Wireless IoT and Cyber-Physical System for Health Monitoring Using Honey Badger Optimized Least-Squares Support-Vector Machine," *Wireless Personal Communications*, vol. 124, no. 4, pp. 3013–3034, Jun. 2022. [Online]. Available: <https://doi.org/10.1007/s11277-022-09500-9>
- [3] B. Ma, Z. Wu, S. Li, R. Benton, D. Li, Y. Huang, M. V. Kasukurthi, J. Lin, G. M. Borchert, S. Tan *et al.*, "Development of a support vector machine learning and smart phone internet of things-based architecture for real-time sleep apnea diagnosis," *BMC Medical Informatics and Decision Making*, vol. 20, no. Suppl 14, p. 298, 2020.
- [4] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," Cryptology ePrint Archive, Paper 2012/144, 2012. [Online]. Available: <https://eprint.iacr.org/2012/144>
- [5] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology – ASIACRYPT 2017*, T. Takagi and T. Peyrin, Eds. Cham: Springer International Publishing, 2017, pp. 409–437.
- [6] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Advances in Cryptology – EUROCRYPT '99*, J. Stern, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 223–238.
- [7] D. Daniel, S. Thomas, and Z. Michael, "Aby - a framework for efficient mixed-protocol secure two-party computation," *Proceedings 2015 Network and Distributed System Security Symposium*, 2015. [Online]. Available: <https://cir.nii.ac.jp/crid/136001143600752256>
- [8] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2.0: Improved Mixed-Protocol secure Two-Party computation," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 2165–2182. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/patra>
- [9] M. Keller, E. Orsini, and P. Scholl, "Mascot: Faster malicious arithmetic secure computation with oblivious transfer," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 830–842. [Online]. Available: <https://doi.org/10.1145/2976749.2978357>
- [10] J. Wang, L. Wu, H. Wang, K.-K. R. Choo, and D. He, "An efficient and privacy-preserving outsourced support vector machine training for internet of medical things," *IEEE Internet of Things Journal*, vol. 8, no. 1, pp. 458–473, 2021.
- [11] X. Liu, R. H. Deng, K.-K. R. Choo, and J. Weng, "An efficient privacy-preserving outsourced calculation toolkit with multiple keys," *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 11, pp. 2401–2414, 2016.
- [12] D. Rathee, M. Rathee, N. Kumar, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "Cryptflow2: Practical 2-party secure inference," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 325–342. [Online]. Available: <https://doi.org/10.1145/3372297.3417274>
- [13] Z. Huang, W. jie Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure Two-Party deep neural network inference," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 809–826. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhichong>
- [14] Y. Fu, Y. Tong, Y. Ning, T. Xu, M. Li, J. Lin, and D. Feng, "Swift: Fast secure neural network inference with fully homomorphic encryption," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 2793–2806, 2025.
- [15] P. Esser, M. Fleissner, and D. Ghoshdastidar, "Non-parametric representation learning with kernels," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 11, pp. 11910–11918, Mar. 2024. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29077>
- [16] X. Ding and F. Yang, "Multi-view randomized kernel classification via nonconvex optimization," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 10, pp. 11793–11801, Mar. 2024. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29064>
- [17] A. Abedsoltan, M. Belkin, and P. Pandit, "Toward large kernel models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 61–78. [Online]. Available: <https://proceedings.mlr.press/v202/abedsoltan23a.html>
- [18] A. X. Yang, M. Robeyns, E. Milsom, B. Anson, N. Schoots, and L. Aitchison, "A theory of representation learning gives a deep generalisation of kernel methods," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 39380–39415. [Online]. Available: <https://proceedings.mlr.press/v202/yang23k.html>
- [19] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction apis," in *Proceedings of the 25th USENIX Conference on Security Symposium*, ser. SEC'16. USA: USENIX Association, 2016, p. 601–618.
- [20] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems*, J. Platt, D. Koller, Y. Singer, and S. Roweis, Eds., vol. 20. Curran Associates, Inc., 2007. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2007/file/013a006f03dbc5392effeb8f18fda755-Paper.pdf
- [21] D. Rathee, M. Rathee, R. K. Kiran Goli, D. Gupta, R. Sharma, N. Chandran, and A. Rastogi, "Sirnn: A math library for secure rnn inference," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1003–1020.
- [22] M. Kelkar, P. H. Le, M. Raykova, and K. Seth, "Secure poisson regression," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 791–808. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/kelkar>
- [23] S. Shalev-Shwartz, Y. Singer, and N. Srebro, "Pegasos: Primal estimated sub-gradient solver for svm," in *Proceedings of the 24th International Conference on Machine Learning*, ser. ICML '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 807–814. [Online]. Available: <https://doi.org/10.1145/1273496.1273598>
- [24] Y. Xie, J. Zhao, B. Zhu, J. Li, R. Li, and K. Xue, "Achieving privacy-preserving outsourced svm training with non-linear kernel," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, 2023, pp. 7115–7120.
- [25] E. Makri, D. Rotaru, N. P. Smart, and F. Vercauteren, "Epic: Efficient private image classification (or: Learning from the masters)," in *Topics in Cryptology – CT-RSA 2019*, M. Matsui, Ed. Cham: Springer International Publishing, 2019, pp. 473–492.
- [26] Y. Liu, Z. Xu, and C. Li, "Distributed online semi-supervised support vector machine," *Information Sciences*, vol. 466, pp. 236–257, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002002551830567X>
- [27] K.-P. Lin and M.-S. Chen, "On the design and analysis of the privacy-preserving svm classifier," *IEEE Transactions on Knowledge and Data Engineering*, vol. 23, no. 11, pp. 1704–1717, 2011.
- [28] Y. Rahulamathavan, S. Veluru, R. C.-W. Phan, J. A. Chambers, and M. Rajarajan, "Privacy-preserving clinical decision support system using gaussian kernel-based classification," *IEEE Journal of Biomedical and Health Informatics*, vol. 18, no. 1, pp. 56–66, 2014.
- [29] B. Xie, T. Xiang, X. Liao, and J. Wu, "Achieving privacy-preserving online diagnosis with outsourced svm in internet of medical things environment," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 4113–4126, 2022.
- [30] R. Wu, B. Wang, and Z. Zhao, "Privacy-preserving medical diagnosis system with gaussian kernel-based support vector machine," *Peer-to-Peer Networking and Applications*, vol. 17, no. 5, pp. 3094–3109, 2024.
- [31] D. Ghada, K. Farinaz, S. Ahmad-Reza, S. Thomas, Z. Shaza, and Z. Michael, "Pushing the communication barrier in secure computation using lookup tables," *Proceedings 2017 Network and Distributed System Security Symposium*, 2017. [Online]. Available: <https://cir.nii.ac.jp/crid/136001144561895040>
- [32] A. Al Badawi, L. Chen, and S. Vig, "Fast homomorphic svm inference on encrypted data," *Neural Comput. Appl.*, vol. 34, no. 18, p. 15555–15573, Sep. 2022. [Online]. Available: <https://doi.org/10.1007/s00521-022-07202-8>
- [33] H. Yang, Q. Zhou, J. Ni, H. Li, and X. Shen, "Accurate image-based pedestrian detection with privacy preservation," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 12, pp. 14494–14509, 2020.

- [34] J. Liang, Z. Qin, J. Ni, X. Lin, and X. Shen, "Efficient and privacy-preserving outsourced svm classification in public cloud," in *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, 2019, pp. 1–6.
- [35] Q. Mao, Y. Chen, P. Duan, B. Zhang, Z. Hong, and B. Wang, "Privacy-preserving classification scheme based on support vector machine," *IEEE Systems Journal*, vol. 16, no. 4, pp. 5906–5916, 2022.
- [36] Y. Rahulamathavan, R. C.-W. Phan, S. Veluru, K. Cumanan, and M. Rajarajan, "Privacy-preserving multi-class support vector machine for outsourcing the data classification in cloud," *IEEE Transactions on Dependable and Secure Computing*, vol. 11, no. 5, pp. 467–479, 2014.
- [37] G. Deng, M. Tang, Y. Xi, and M. Zhang, "Privacy-preserving online medical prediagnosis training model based on soft-margin svm," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 2072–2084, 2023.
- [38] C. Hu, C. Zhang, D. Lei, T. Wu, X. Liu, and L. Zhu, "Achieving privacy-preserving and verifiable support vector machine training in the cloud," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 3476–3491, 2023.
- [39] X. Liu, R. H. Deng, K.-K. R. Choo, and Y. Yang, "Privacy-preserving outsourced support vector machine design for secure drug discovery," *IEEE Transactions on Cloud Computing*, vol. 8, no. 2, pp. 610–622, 2020.
- [40] J. Platt, "Sequential minimal optimization: A fast algorithm for training support vector machines," Microsoft, Tech. Rep. MSR-TR-98-14, April 1998.
- [41] S. Park, J. Byun, J. Lee, J. H. Cheon, and J. Lee, "He-friendly algorithm for privacy-preserving svm training," *IEEE Access*, vol. 8, pp. 57 414–57 425, 2020.
- [42] F.-J. González-Serrano, Ángel Navia-Vázquez, and A. Amor-Martín, "Training support vector machines with privacy-protected data," *Pattern Recognition*, vol. 72, pp. 93–107, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320317302406>
- [43] G. Asharov, Y. Lindell, T. Schneider, and M. Zohner, "More efficient oblivious transfer and extensions for faster secure computation," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*, ser. CCS '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 535–548. [Online]. Available: <https://doi.org/10.1145/2508859.2516738>
- [44] V. Kolesnikov and R. Kumaresan, "Improved ot extension for transferring short secrets," in *Advances in Cryptology – CRYPTO 2013*, R. Canetti and J. A. Garay, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 54–70.
- [45] R. Canetti, "Security and composition of multiparty cryptographic protocols," *Journal of CRYPTOLOGY*, vol. 13, pp. 143–202, 2000.
- [46] O. Goldreich, S. Micali, and A. Wigderson, *How to play any mental game, or a completeness theorem for protocols with honest majority*. New York, NY, USA: Association for Computing Machinery, 2019, p. 307–328. [Online]. Available: <https://doi.org/10.1145/3335741.3335755>
- [47] Y. Lindell, *How to Simulate It – A Tutorial on the Simulation Proof Technique*. Cham: Springer International Publishing, 2017, pp. 277–346. [Online]. Available: https://doi.org/10.1007/978-3-319-57048-8_6
- [48] S. Halevi and V. Shoup, "Faster homomorphic linear transformations in helib," in *Advances in Cryptology – CRYPTO 2018*, H. Shacham and A. Boldyreva, Eds. Cham: Springer International Publishing, 2018, pp. 93–120.